Name: Long Tran

Date: 11/11/2026

Course: Foundations of Python Programming

GitHub URL: https://github.com/LT-Mesoda/IntroToProg-Python-Mod05

Assignment 5 – Introduction to Python

Introduction

The fifth module establishes the next step which introduces methods to handle and store larger amounts of data. Data structures are introduced, primarily lists and dictionaries, which use indexes and keys to as a way to establish the different categories of data. These concepts introduce some dimensionality to the data. Another important aspect introduced in this module is handling user errors by accounting for false entries. This builds on conditional logic with try-except functions and using JavaScript Object Notation (JSON). Finally, the aspect of collaboration and shared code spaces are introduced as well. The assignment will be linked above.

Topic

The constants and variables are established at the beginning of the script. These variables will store the user entered data, as seen in **Error! Reference source not found.**.

```
import json

# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-----------------------------------------
'''
# Define the Data Constants
FILE_NAME: str = "Enrollments.json"

# Define the Data Variables and constants
student_first_name: str = ''  # First name of a student entered by the user.
student_last_name: str = ''  # Last name of a student entered by the user.
course_name: str = ''  # Name of a course entered by the user.
file = None  # Holds a reference to an opened file.
menu_choice: str  # Hold the choice made by the user.
student_data: dict = []  # Dictionary for student data
students: list = []  # a table of student data
```

Figure 1 Constants and Variables in the Script

Some error handling is used, prompting an error message if the necessary JSON file is not present in the reference folder. If there is a file, then it will load the file into a variable, as seen in Figure 2.

```
33    # When the program starts, read the file data into a list of lists (table)
34    # Extract the data from the file
35    try:
36        file = open(FILE_NAME, "r")
37        # Load JSON file with module
38        students = json.load(file)
39        for item in students:
40            print(f"FirstName:{item['FirstName']}, LastName:{item['LastName']}")
41        file.close()
42    except FileNotFoundError as e:
43        print("Text file must exist before running this script!\n")
44        print("-- Technical Error Message -- ")
45        print(e, e.__doc__, type(e), sep='\n')
46    except Exception as e:
47        print("There was a non-specific error!\n")
48        print("-- Technical Error Message -- ")
49        print(e, e.__doc__, type(e), sep='\n')
50    finally:
51        if file is not None and file.closed == False:
52            file.close()
53
54    # Check if a file object exists and is still open
55    if file is not None and file.closed == False:
56        file.close()
```

Figure 2 Opening JSON file and error message if it is not present in working directory

The while loop goes through the established process that:

- Requires the user to enter in the name of a student or multiple students for a course.
- Show the data for the entries.
- Save the list of lists to a specific file named Enrollments.csv.
- End the while loop and exit the program.

Every time the process loops, the program prints the menu showing the options and requires the user to make a choice. In addition to user input, if the entry value does not fall under acceptable criteria – the name has a number – then an error message will be raised. This is detected by the function is.alpha which detects if the input has a number. The inputs are then saved into a dictionary designated as student_data as seen in Figure 3.

```
# Present and Process the data
while (True):

    # Present the menu of choices
    print(MENU)
    menu_choice = input("Enter a menu option (1-4): ")

    # Input user data
    if menu_choice == "1":  # This will not work if it is an integer!
        try:
            student_first_name = input("Enter the student's first name: ")
            if not student_first_name.isalpha():
                raise ValueError("The first name should not contain numbers.")

            student_last_name = input("Enter the student's last name: ")
            if not student_last_name.isalpha():
                raise ValueError("The last name should not contain numbers.")

            course_name = input("Enter the name of the course: ")

            student_data = [student_first_name,student_last_name,course_name]
            student_data = {"FirstName":student_data[0],
                            "LastName":student_data[1],
                            "CourseName":student_data[2]}
            students.append(student_data)
            print(
                f"You have registered {student_first_name}"
                  f" {student_last_name} for {course_name}."
            )
        except ValueError as e:
            print(e)  # Prints the custom message
            print("-- Technical Error Message -- ")
            print(e.__doc__)
            print(e.__str__())
        except Exception as e:
            print("There was a non-specific error!\n")
            print("-- Technical Error Message -- ")
            print(e, e.__doc__, type(e), sep='\n')

        finally:
            if file is not None and file.closed == False:
                file.close()
        continue
```

Figure 3 While loop requiring user input and error message if input fails criteria

Once the data is entered, the data is processed and a custom message is displayed when the second menu choice is selected. The data entries are prompted on the console. The data is then saved when the user designates for the third option provided by the menu. This requires the file to be open and written, the data under the list ‘students’ is “dumped” using the dump function and saved. A try-except function is applied here as well, in the scenario where the JSON file is not present. Once the data process is done, the user is prompted to end the procedure with the final step with entry 4, which ends the program, shown in Figure 4.

```
113         # Save the data to a file
114         elif menu_choice == "3":
115             try:
116                 file = open(FILE_NAME, "w")
117                 json.dump(students,file)
118 
119             # Check if a file object exists and is still open
120                 if file is not None and file.closed == False:
121                     file.close()
122 
123                 print("The following data was saved to file!")
124                 for student in students:
125                     print(f'{student["FirstName"]},{student["LastName"]},\
126                           {student["CourseName"]}')
127                     continue
128             except FileNotFoundError as e:
129                 print("Text file must exist before running this script!\n")
130                 print("-- Technical Error Message -- ")
131                 print(e, e.__doc__, type(e), sep='\n')
132             except Exception as e:
133                 print("There was a non-specific error!\n")
134                 print("-- Technical Error Message -- ")
135                 print(e, e.__doc__, type(e), sep='\n')
136             finally:
137                 if file is not None and file.closed == False:
138                     file.close()
139 
140         # Stop the loop
141         elif menu_choice == "4":
142             break  # out of the loop
143         else:
144             print("Please only choose option 1, 2, 3, or 4")
145 
146     print("Program Ended")
147 
```

Figure 4 Process of appending and saving user data into present JSON file

Summary

The module shows methods to handle and store larger amounts of data with lists and dictionaries and using JavaScript Object Notation (JSON). With large data processing, errors can occur that can compound later on in the process. Therefore, the module also includes handling user errors to accounting for false entries with the use of conditional logic with try-except functions). The work will be displayed in a GitHub location.